INSTITUT TEKNOLOGI DEL

MATERI TUGAS

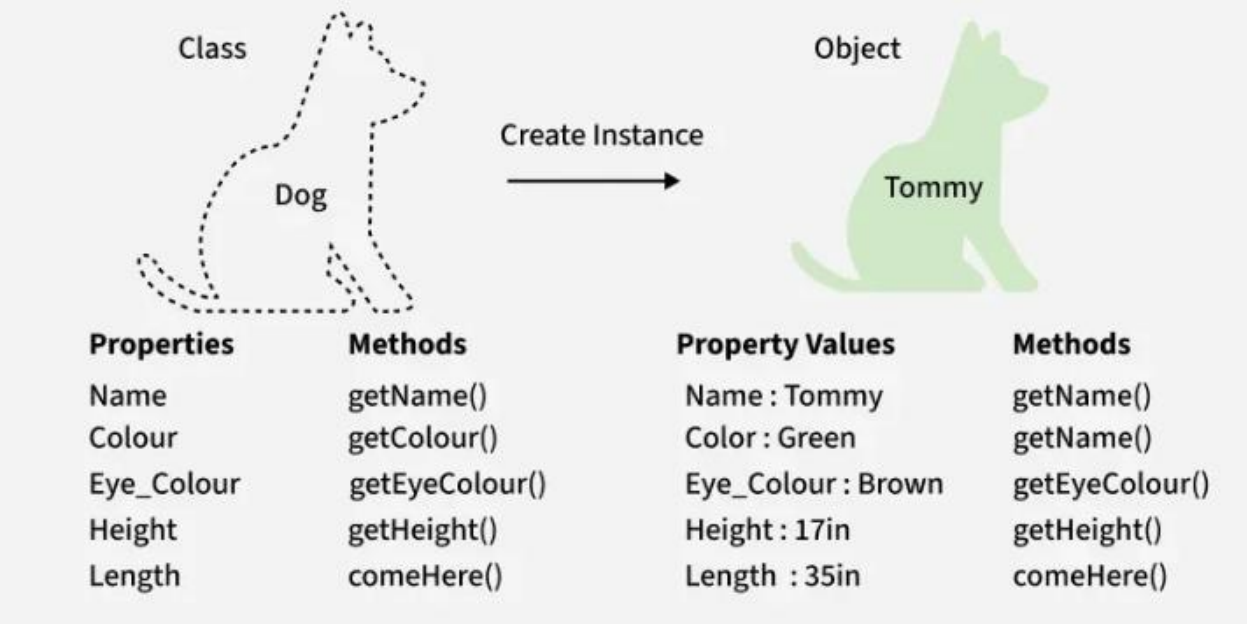
Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2026/2027

Tanggal / Sesi	:	Rabu, 23 September 2026
Semester	:	Ganjil
Mata Kuliah	:	Pemrograman Berorientasi Objek
Minggu/Sesi	:	03/ 02
Aktivitas	:	Classes and Objects, Inheritance
Durasi	:	110 Menit
Setoran	:	Softfile
Batas Pengumpulan	:	Akhir Sesi
Tujuan	:	- Mahasiswa mampu membuat class dan menginstasiasi objek dari sebuah kelas dan inheritance. - Melatih *computational thinking*: memformulasikan masalah → merancang solusi → menulis kode → memverifikasi output

Class dan Object

Dalam java, Class dan Object merupakan konsep dasar Pemograman Berorientasi Objek (OOP) yang digunakan untuk merepresentasikan konsep dan entitas dunia nyata yang dapat kita lihat dari ilustrasi di bawah.



Dari ilustrasi di atas kita dapat melihat bahwa **class `Dog`** berperan sebagai *blueprint* (cetak biru) atau template yang mendefinisikan:

- Properties (atribut/fields) → seperti `name`, `colour`, `eye_colour`, `height`, dan `length`.
- Methods (fungsi/perilaku) → seperti `getName()`, `getColour()`, `getEyeColour()`, `getHeight()`, dan `comeHere()`.

Class hanya mendeskripsikan apa yang dimiliki (atribut) dan dapat dilakukan (perilaku) oleh suatu objek, tetapi belum merupakan objek nyata.

Kemudian, ketika kita membuat instance dari class `Dog`, misalnya objek bernama **`Tommy`** maka,

- Objek tersebut akan memiliki nilai konkret untuk setiap property (contoh: `name = "Tommy"`, `colour = "Green"`, `eye_colour = "Brown"`, `height = 17in`, `length = 35in`).
- Objek tersebut juga dapat menjalankan method-method yang sudah didefinisikan di dalam class `Dog`.

Class

Sebuah **Class** di java merupakan sebuah template (*blueprint*) yang digunakan untuk membuat entitas benda nyata yang disebut Object, yang memiliki karakteristik dan properti yang sama. Mari kita lihat bagaimana penggunaan dasar dari Class di java. Kita dapat melihat pada code di bawah.

```java
class Student {
    int id;
    String n;

    public Student(int id, String n) {
        this.id = id;
        this.n = n;
    }
}


public class Main {
    public static void main(String[] args) {

        Student s1 = new Student(10, "Alice");

        System.out.println(s1.id);
        System.out.println(s1.n);
    }
}
```

Output:

```
run:
10
Alice
```

```
access_modifier class<class_name> { //merupakan deklarasi class di java
    data member;
    method;
    constructor;
    nested class;
    interface
    ;
}
```

Secara umum, deklarasi Class dapat mencakup komponen-komponen secara berurutan yaitu, Modifiers, kata kunci Class, nama Class, Superclass(jika ada), Interfaces(jika ada), Body, Fields dan Constructors.

Object

Objek adalah instance dari sebuah class yang dibuat untuk menggunakan atribut dan method dari class tersebut. Sebuah program Java biasanya membuat banyak objek yang saling berinteraksi dengan memanggil method. Sebuah objek terdiri dari *State* (attributes), *Behavior* (methods) dan *Identity* (nama unik pada sebuah objek).



Objek berhubungan dengan hal-hal yang ditemukan di dunia nyata.Misalnya, sebuah program grafis mungkin memiliki objek seperti ‘lingkaran’, ‘persegi’ dan ‘menu’ atau sebuah program belanja online mungkin memiliki objek seperti ‘produk’, ‘keranjang belanja’, ‘pelanggan’.

Ketika sebuah objek dari suatu class dibuat, maka class tersebut dikatakan telah di instansiasi. Semua instans memiliki atribut dan perilaku yang sama dengan class tersebut. Namun nilai attribute, yaitu statusnya bersifat unik untuk objek. Kalau kita deklarasikan variabel seperti (nama tipe), itu memberitahu kompiler bahwa variabel tersebut akan menyimpan data dengan tipe tertentu.

- Untuk tipe primitif (seperti int, double), deklarasi langsung menyediakan memori untuk menyimpan nilainya.
- Untuk variabel referensi (seperti `Dog tuffy`), tipenya harus berupa nama Class nyata. Kita tidak bisa membuat objek langsung dari kelas abstrak atau interface.

Contoh: `Dog Tuffy`

Disini Tuffy hanya referensi kosong. Nilainya tidak akan ditentukan (`null`) hingga sebuah objek benar-benar dibuat dan di tetapkan padanya.

Inisialisasi Objek

Operator `new` membuat instance Class dengan mengalokasikan memori untuk objek baru dan mengembalikan referensi ke memori ke memori tersebut. Operator `new` juga memanggil konstruktor Class. Kita dapat melihat contoh code berikut

```
public class Dog {
    String name;
    String breed;
    int age;
    String color;

    // Constructor
    public Dog(String name, String breed, int age,
               String color)
    {
        this.name = name;
        this.breed = breed;
        this.age = age;
        this.color = color;
    }

    // method 1
    public String getName() {
      return name;
    }

    // method 2
    public String getBreed() {
      return breed;
    }

    // method 3
    public int getAge() {
      return age;
    }
```

```
    // method 4
    public String getColor() {
      return color;
    }

    @Override public String toString()
    {
        return ("Name is: " + this.getName()
                + "\nBreed, age, and color are: "
                + this.getBreed() + "," + this.getAge()
                + "," + this.getColor());
    }

    public static void main(String[] args)
    {
        Dog tuffy
            = new Dog("tuffy", "papillon", 5, "white");
        System.out.println(tuffy.toString());
    }
}
```

Output:

```
run:
Name is: tuffy
Breed, age, and color are: papillon,5,white
```

Program di atas mendefinisikan kelas `Dog` dengan variabel instance `name, breed, age`, dan `color`. Nilai awal diberikan melalui konstruktor, sedangkan method getter digunakan untuk mengambil data dari objek. Method `toString`() dioverride agar saat objek dicetak, informasi ditampilkan secara jelas. Pada `main`(), dibuat objek `Dog` bernama tuffy dengan ras papillon, umur 5 tahun, dan warna putih, lalu detail objek tersebut ditampilkan ke layar.

Inisialisasi Objek Menggunakan Method/Function

Sebuah objek di Java juga dapat diinisialisasi melalui method atau function khusus. Caranya adalah dengan terlebih dahulu membuat objek menggunakan operator new, kemudian memanggil method yang bertugas mengatur nilai atribut atau variabel instance dari objek tersebut. Metode ini sering disebut initializer method atau setter method.

```java
public class Geeks {

    static String name;
    static float price;

    static void set(String n, float p) {
        name = n;
        price = p;
    }

    static void get()
    {
        System.out.println("Software name is: " + name);
        System.out.println("Software price is: " + price);
    }

    public static void main(String args[])
    {
        Geeks.set("Visual studio", 0.0f);
        Geeks.get();
    }
}
```

Output:

```
run:
Software name is: Visual studio
Software price is: 0.0
```

Cara Membuat Objek dari Sebuah Class

Ada 4 cara untuk membuat objek di java. Meskipun kata kunci new merupakan cara uatama untuk membuat objek, metode lain juga secara internal bergantung pada kata kunci new untuk membuat instans.

1. **Menggunakan new Keyboard**

   Ini merupakan cara paling umum dan umum untuk membuat objek di class.

   ```
   // membuat object
   Test t = new Test();
   ```

2. **Menggunakan Reflection**

   Reflection merupakan fitur hebat di java yang memungkin program untuk memeriksa dan mengubah struktur serta perilakunya sendiri saat runtime.

```
class Student { //Asumsikan kelas Student sudah ada (tunjukkan definisinya)
    public Student() {}
}

public class Main {
    public static void main(String[] args) {
        try {
            Class<?> c = Class.forName("Student");
            Student s2 = (Student) c.getDeclaredConstructor().newInstance();
            System.out.println("Object created: " + s2);
        } catch (ClassNotFoundException e) {
            System.err.println("Class not found!");
        } catch (NoSuchMethodException e) {
            System.err.println("No default constructor!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3. Menggunakan clone() method

Metode `clone()` terdapat di dalam class objek. Metode ini membuat dan mengembalikan salinan objek.

```
class Geeks implements Cloneable {
    @Override //method 1
    protected Object clone()
        throws CloneNotSupportedException
    {
        return super.clone();// Super() mengacu ke parent class
    }
    String name = "GeeksForGeeks";

    public static void main(String[] args)// Method 2
    {
        Geeks o1 = new Geeks();

        try {
            Geeks o2 = (Geeks)o1.clone();
            System.out.println(o2.name);
        }
        catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
run:
GeeksForGeeks
```

4. Deserialization

Deserialisasi merupakan teknik membaca objek dari status tersimpan dalam sebuah class.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class Main {
    public static void main(String[] args)
    {
        // Serialization
        try (ObjectOutputStream out
             = new ObjectOutputStream(
                 new FileOutputStream("student.ser"))) {
            out.writeObject(new Student("Alice"));
        }
        catch (IOException e) {
            e.printStackTrace();
        }
        // Deserialization
        try (ObjectInputStream in = new ObjectInputStream(
                 new FileInputStream("student.ser"))) {
            Student s = (Student)in.readObject();
            System.out.println(s);
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
run:
Student: Alice
```

Membuat Beberapa Objek dengan Satu Jenis

Secara real-time, kita membutuhkan objek yang berbeda dari suatu class dengan metode yang berbeda pula. Membuat sejumlah referensi untuk menyimpannya bukanlah praktik yang baik, oleh karena itu kita mendeklarasikan variabel referensi statis dan menggunakannya kapan pun diperlukan. Dalam sistem pewarisan, kita menggunakan variabel referensi kelas induk untuk menyimpan objek subclass. Dalam hal ini, kita dapat beralih ke objek subkelas yang berbeda menggunakan variabel referensi yang sama.

```
class Animal {
}
class Dog extends Animal {
}
class Cat extends Animal {
}
public class Test {
    Animal obj = new Dog();  // Menggunakan Dog object
    obj = new Cat();  // Menggunakan Cat object
}
```

Membuat Objek Anonymous

Objek anonym adalah objek yang diinstansiasi tanpa menyimpan referensinya dalam variabel. Objek ini digunakan untuk operasi satu kali (mis, pemanggilan metode) dan langsung dibuang setelah digunakan.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class Main extends Application {
    public static void main(String[] args) { launch(args); }
    @Override public void start(Stage stage)
    {
        Button button = new Button("Click Me");
        button.setOnAction(
            event
            -> System.out.println(
                "Button clicked!")); // Anonymous object melalui lambda

        StackPane root = new StackPane(button);
        stage.setScene(new Scene(root, 300, 200));
        stage.show();
    }
}
```

Class Object di Java

Dalam pemrograman berorientasi objek (OOP) di Java, Class **`Object`** merupakan class induk (superclass) dari semua class yang ada. Artinya, setiap class yang kita buat di Java secara langsung atau tidak langsung mewarisi class `Object`, meskipun kita tidak menuliskan `extends Object` secara eksplisit. Class `Object` berada di dalam paket **`java.lang`**, yang otomatis diimpor ke setiap program Java. Karena itu, semua kelas yang dibuat oleh programmer secara otomatis akan memiliki perilaku dan metode dasar yang berasal dari kelas `Object`.

Sebagai akar dari hierarki kelas di Java, kelas `Object` menyediakan metode-metode penting dan umum yang dapat digunakan atau dioverride oleh setiap kelas turunan. Beberapa metode utama yang disediakan antara lain:

1. toString() method

Metode `toString()` menyediakan representasi berupa `String` dari sebuah objek dan digunakan untuk mengonversi objek menjadi `String`. Secara default, metode `toString()` pada kelas `Object` akan mengembalikan sebuah string yang terdiri dari nama class tempat objek tersebut dibuat, karakter tanda `@`, dan representasi heksadesimal (tanpa tanda) dari nilai hash code objek tersebut.

```
public class Student {

    @Override
    public String toString() {
        return "Student object";
    }

    public static void main(String[] args) {
        Student s = new Student();          // membuat objek Student
        System.out.println(s.toString());   // memanggil toString()
    }
}
```

2. hashCode() method

Untuk setiap objek, JVM (Java Virtual Machine) menghasilkan sebuah angka unik yang disebut `hashcode`. Metode ini mengembalikan bilangan integer yang berbeda untuk setiap objek yang berbeda. Kegunaan Metode `hashCode()`:

- Metode ini mengembalikan nilai hash yang digunakan untuk mencari objek dalam sebuah koleksi.
- JVM menggunakan `hashCode()` saat menyimpan objek ke dalam struktur data berbasis hashing seperti `HashSet`, `HashMap`, dan `Hashtable`.
- Keuntungan utama menyimpan objek berdasarkan hash code adalah proses pencarian menjadi lebih cepat dan efisien.

```
public class Student {

    int roll;

    @Override
    public int hashCode() {
        return roll;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;

        Student s2 = new Student();
        s2.roll = 102;

        System.out.println("HashCode s1: " + s1.hashCode());
        System.out.println("HashCode s2: " + s2.hashCode());
    }
}
```

3. equals(object obj) method

Metode `equals()` digunakan untuk membandingkan objek yang diberikan dengan objek saat ini. Biasanya**,** disarankan untuk meng-override metode ini ketika kita ingin menentukan kondisi kesetaraan khusus untuk objek buatan sendiri.

```java
public class Student {

    int roll;

    @Override
    public boolean equals(Object o) {
        if (o instanceof Student) {
            return this.roll == ((Student) o).roll;
        }
        return false;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;

        Student s2 = new Student();
        s2.roll = 101;

        Student s3 = new Student();
        s3.roll = 102;

        System.out.println("s1.equals(s2): " + s1.equals(s2));
        System.out.println("s1.equals(s3): " + s1.equals(s3));
    }
}
```

4. finalize() method

Metode `finalize()` dipanggil tepat sebelum sebuah objek dikumpulkan oleh Garbage Collector. Metode ini dipanggil oleh Garbage Collector pada sebuah objek ketika ia menentukan bahwa tidak ada lagi referensi terhadap objek tersebut.Kita dapat meng-override metode `finalize()` untuk melepaskan resource sistem, melakukan aktivitas pembersihan (clean-up), dan meminimalkan kebocoran memori.

```
public class Geeks {
    public static void main(String[] args) {

        Geeks t = new Geeks();
        System.out.println(t.hashCode());

        t = null;

        System.gc();

        System.out.println("end");
    }

    @Override protected void finalize()
    {
        System.out.println("finalize method called");
    }
}
```

5. getClass() method

Metode `getClass()` mengembalikan class object dari objek `this` dan digunakan untuk mendapatkan kelas sebenarnya (runtime class) dari objek tersebut. Metode ini juga dapat digunakan untuk mendapatkan metadata dari kelas tersebut. Class object yang dikembalikan adalah objek yang dikunci (locked) oleh static synchronized methods dari kelas yang direpresentasikan. Karena metode ini bertipe final, maka **tidak dapat dioverride**.

```
public class Geeks {
    public static void main(String[] args)
    {
        Object o = new String("GeeksForGeeks");
        Class c = o.getClass();
        System.out.println("Class of Object o is: "
                           + c.getName());
    }
}
```

6. clone()method

Metode `clone()` membuat dan mengembalikan objek baru yang merupakan salinan dari objek saat ini.

```java
public class Book implements Cloneable {
    private String t;
    public Book(String t) {
        this.t = t;
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    public String getTitle() {
        return t;
    }

    public static void main(String[] args) {
        try {
            Book b1 = new Book("Java Basics");
            Book b2 = (Book) b1.clone();

            System.out.println("Judul buku b1: " + b1.getTitle());
            System.out.println("Judul buku b2: " + b2.getTitle());
            System.out.println("Apakah b1 dan b2 objek yang sama? " + (b1 == b2));

        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

7. wait(), notif(), notifAll() (Concurrency methods)

Metode `wait()`, `notify()`, dan `notifyAll()` adalah metode concurrency di Java yang digunakan untuk sinkronisasi antar *thread*, khususnya saat beberapa *thread* berbagi *resource* yang sama (misalnya variabel atau objek). Metode-metode ini adalah bagian dari class Object (bukan Thread), sehingga semua objek di Java punya metode ini secara bawaan. Tapi penggunaannya hanya bisa dilakukan di dalam blok synchronized atau metode synchronized, karena tujuannya adalah mengatur thread yang mengunci (lock) objek tersebut.

```java
public class Book implements Cloneable {
    private String t;
    private String a;
    private int y;

    public Book(String t, String a, int y)
    {
        this.t = t;
        this.a = a;
        this.y = y;
    }

    @Override public String toString()
    {
        return t + " by " + a + " (" + y + ")";
    }

    @Override public boolean equals(Object o)
    {
        if (o == null || !(o instanceof Book)) {
            return false;
        }
        Book other = (Book)o;
        return this.t.equals(other.getTitle())
            && this.a.equals(other.getAuthor())
            && this.y == other.getYear();
    }

    @Override public int hashCode()
    {
        int res = 17;
        res = 31 * res + t.hashCode();
        res = 31 * res + a.hashCode();
        res = 31 * res + y;
        return res;
    }

    @Override public Book clone()
    {
        try {
            return (Book)super.clone();
        }
        catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
```

```
    @Override protected void finalize() throws Throwable
    {
        System.out.println("Finalizing " + this);
    }

    public String getTitle() { return t; }
    public String getAuthor() { return a; }
    public int getYear() { return y; }

    public static void main(String[] args)
    {
        Book b1 = new Book(
            "The Hitchhiker's Guide to the Galaxy",
            "Douglas Adams", 1979);
        System.out.println(b1);

        Book b2 = b1.clone();
        System.out.println(b2);

        System.out.println("b1 equals b2: "
                           + b1.equals(b2));

        System.out.println("b1 hash code: "
                           + b1.hashCode());
        System.out.println("b2 hash code: "
                           + b2.hashCode());
        b1 = null;
        System.gc();
    }
}
```

Dengan adanya kelas `Object`, Java memiliki struktur pewarisan yang konsisten, sehingga setiap objek dapat diperlakukan secara umum sebagai turunan dari kelas yang sama. Hal ini memudahkan pengelolaan objek dalam berbagai konteks seperti koleksi (*Collection*), pemanggilan metode generik, dan mekanisme pemrograman berbasis objek secara keseluruhan.

Inheritance

Inheritance atau pewarisan adalah salah satu konsep utama dalam pemrograman berorientasi objek (Object-Oriented Programming/OOP) yang memungkinkan sebuah kelas mewarisi sifat (atribut) dan perilaku (metode) dari kelas lain. Dalam Java, inheritance digunakan untuk menciptakan hubungan **"**IS-A**"** antara dua kelas, yaitu kelas anak (*subclass*) dan kelas induk (*superclass*). Subclass dapat menggunakan kembali kode yang telah ditulis di superclass, sehingga mengurangi duplikasi kode dan memudahkan pemeliharaan program. Melalui inheritance, subclass tidak hanya dapat menggunakan anggota yang dimiliki superclass, tetapi juga dapat menambahkan anggota baru atau memodifikasi perilaku yang ada melalui mekanisme overriding. Terdapat beberapa type inheritance di java yaitu sebagai berikut.

1. Singel Inheritance

Single inheritance adalah bentuk pewarisan paling sederhana di mana sebuah class anak (*child class*) hanya mewarisi dari satu class induk (*parent class*). Dalam konsep ini, *child class* dapat menggunakan semua properti dan method yang dimiliki *parent class*, sekaligus menambahkan fitur miliknya sendiri. Kita dapat melihat pada contoh code berikut.

```
class Vehicle { //Super class
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}

// Subclass
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

public class Test {
    public static void main(String[] args) {
        Car obj = new Car();
    }
}
```

2. Multilevel Inheritance

Multilevel inheritance terjadi ketika pewarisan dilakukan secara bertingkat, di mana sebuah *class* mewarisi dari *class* lain, dan *class* berikutnya mewarisi lagi dari *class* tersebut. Kita dapat melihat pada contoh code berikut.

```
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}
class FourWheeler extends Vehicle {
    FourWheeler() {
        System.out.println("4 Wheeler Vehicles");
    }
}
class Car extends FourWheeler {
    Car() {
        System.out.println("This 4 Wheeler Vehicle is a Car");
    }
}

public class Geeks {
    public static void main(String[] args) {
        Car obj = new Car();
    }
}
```

3. Hierarchical Inheritance

Hierarchical inheritance digunakan saat satu class induk memiliki banyak class anak. Semua class anak mewarisi atribut dan method yang sama dari class induk, sehingga kode yang umum hanya ditulis sekali di parent class. Kita dapat melihat pada contoh code berikut.

```java
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}

class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

class Bus extends Vehicle {
    Bus() {
        System.out.println("This Vehicle is Bus");
    }
}

public class Test {
    public static void main(String[] args) {
        Car obj1 = new Car();
        Bus obj2 = new Bus();
    }
}
```

4. Multiple Inheritance

Multiple inheritance tidak bisa dilakukan secara langsung antar class di Java karena bisa menimbulkan konflik pewarisan, tetapi bisa dilakukan lewat *interface*. Dengan *interface*, satu class dapat mengimplementasikan banyak interface sekaligus dan mewarisi perilaku dari semuanya. Kita dapat melihat pada contoh code berikut.

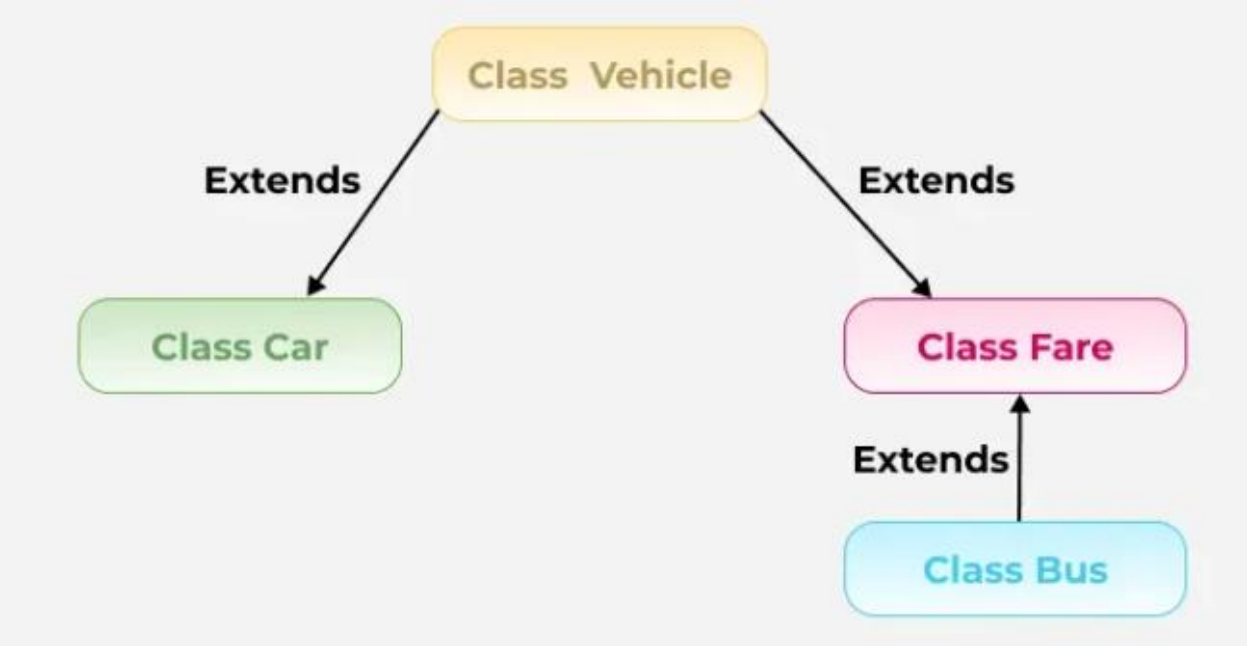
```java
interface LandVehicle {
    default void landInfo() {
        System.out.println("This is a LandVehicle");
    }
}
interface WaterVehicle {
    default void waterInfo() {
        System.out.println("This is a WaterVehicle");
    }
}
```

```java
class AmphibiousVehicle implements LandVehicle, WaterVehicle {
    AmphibiousVehicle() {
        System.out.println("This is an AmphibiousVehicle");
    }
}

public class Test {
    public static void main(String[] args) {
        AmphibiousVehicle obj = new AmphibiousVehicle();
        obj.waterInfo();
        obj.landInfo();
    }
}
```

5. Hybrid Inheritance

Hybrid inheritance merupakan kombinasi dari beberapa jenis inheritance sekaligus, misalnya gabungan hierarchical dan multiple inheritance. Karena Java tidak mendukung multiple inheritance secara langsung, hybrid inheritance di Java biasanya dilakukan dengan menggabungkan pewarisan antar class dengan implementasi beberapa interface. Pendekatan ini memungkinkan desain program yang lebih kompleks dan fleksibel, tetapi juga harus dirancang dengan hati-hati agar tidak membingungkan dan sulit dirawat.



TUGAS:

Soal 1 – Fill in the Blank

Petunjuk: Lengkapilah kode berikut agar dapat berjalan dengan benar dan menampilkan.

Output:

```
Nama: Budi

NIM: 12345
```

```
class Mahasiswa {
    String nama;
    String nim;
}

public class Main {
    public static void main(String[] args) {
        Mahasiswa m1 = new _____();
        m1.nama = _____;
        m1.nim = _____;

        System.out.println("Nama: " + _____);
        System.out.println("NIM: " + _____);
    }
}
```

Soal 2 – Studi Kasus Sederhana

Kasus: Sebuah kampus memiliki dua jenis mahasiswa: Mahasiswa Biasa dan Mahasiswa Aktif.

- Mahasiswa biasa memiliki `nama` dan `nim`.
- Mahasiswa aktif memiliki tambahan `semester` dan dapat `belajar()`.

Tugas:

1. Buat class `Mahasiswa` dan `MahasiswaAktif` (`extends Mahasiswa`)
2. Isi data objek `MahasiswaAktif`

Output:

```
run:
Nama: Dina
NIM: 123456
Dina sedang belajar di semester 5
```